AVINASH PAL • LINUX SYSTEMS ENGINEERING

Think Like a Linux Storage Engineer

From Linux I/O Internals to Filesystems, blk-mq, RAID, NVMe and Storage Debugging

TOPIC: STORAGE & I/O

TABLE OF CONTENTS (FULL EBOOK SYLLABUS)

1. Thinking Like a Linux Storage Engineer
2. The Linux Storage Stack: VFS to Hardware
3. VFS, Page Cache, and the Block Layer
4. Stacking Storage: DM, LVM, md RAID, and Multipath
5. ext4 and XFS: Journaling Filesystems in Production
6. Btrfs and Copy-on-Write Filesystem Architecture
7. NVMe, PCIe, and the Local Fast Path
8. SCSI, SAS, SATA, and Network Storage Fabrics
9. Performance, Benchmarking with fio, and bpftrace
10. Reliability, Silent Corruption, and Interview Mastery

CHAPTER 1 EXCERPT & TECHNICAL NOTES

Author: Avinash Pal | Target: Linux 6.x LTS

1. The I/O Fast Path: From write() to NVMe Doorbell

When a user program issues write(), the data is first placed in the kernel Page Cache folios.
Writeback threads (wb_workfn) flush dirty folios into BIOs submitted to the block layer.
blk-mq allocates hardware queue slots and issues a doorbell write on the PCIe NVMe controller.
Completion is signaled via MSI-X interrupt, returning up through the block completion handler.

2. Key Storage Debugging Tools & Formulas

• fio --name=randwrite --ioengine=libaio --iodepth=64 --rw=randwrite --bs=4k --direct=1
• bpftrace -e 'kprobe:blk_account_io_done { @lat_us = hist(arg1); }'
• nvme smart-log /dev/nvme0n1 # Monitor write endurance and throttling warnings
• iostat -xz 1 # Observe %util, await, r_await, w_await, and queue size (aqu-sz)

3. Top Interview Conceptual Question

Q: Why does blk-mq use two layers of queues (software queues and hardware dispatch queues)?
A: To eliminate per-CPU cache ping-pong locks when thousands of parallel threads issue I/O simultaneously to ultra-high IOPS NVMe controllers.